Practical : 2	Implementation and Time analysis of linear and binary search algorithm.
Date : 08/07/2026	

Aim: To implement and analyze **Linear Search and Binary Search** algorithms.

A)LINEAR SEARCH :-

Objective :- To implement the Linear Search algorithm in C++ using an array, and to analyze its performance across best, worst, and average-case scenarios.

Linear Search Algorithm

1. Start.
2. Read the array and target element.
3. Start from the first element.
4. Compare each element with the target.
5. If the element matches, return its index.
6. Otherwise, continue to the next element.
7. If the end is reached, return -1.
8. Display the result.
9. Stop.

Program :-

```
#include <iostream>

using namespace std;

// Function for Linear Search

int linearSearch(int arr[], int size, int target) {

   for (int i = 0; i < size; i++) {

      if (arr[i] == target) {

         return i;
```

```
        }
    }
    return -1;
}
int main()
int arr[] = {12, 45, 7, 23, 56, 90, 3};
int size = sizeof(arr) / sizeof(arr[0]);
int target;
cout << "Enter the number you want to search for: ";
 cin >> target;
int result = linearSearch(arr, size, target);
if (result != -1) {
        cout << "Element found at index: " << result << endl;
    }
    else {
        cout << "Element not found in the array." << endl;
    }
return 0;
}
```

Output:

```
Output
Enter the number you want to search for: 23
Element found at index: 3


=== Code Execution Successful ===
```

Analysis:-

1. Time Complexity Analysis

The execution time of Linear Search relies on the number of comparisons made within the forloop relative to the array size **n**.

- **Best Case: O(1)**

  This happens when the target element matches the very first element of the array **(arr[0]).** The loop terminates instantly on its first pass, executing exactly 1 comparison.

- **Worst Case: O(n)**

  This happens if the target element is located at the very end of the array **(arr[size-1])** or is completely missing from it. The loop must iterate over every single item, executing exactly **n** comparisons.

- **Average Case: O(n)**

  Assuming the target element has an equal probability of being located at any index, the algorithm will inspect roughly half the array ($\frac{n+1}{2}$ elements) before finding it. Dropping constants leaves a linear growth rate.

2. Space Complexity Analysis

- **Space Complexity: O(1)**

  The algorithm performs its operations completely in place inside the existing array. The only extra memory allocated during execution is for a few basic variables (i and size). Because this memory allocation does not grow with the array size, it requires constant auxiliary space.

Conclusion :-

Through this practical, we observed that Linear Search is a simple andstraightforward searching technique. Its primary benefit is flexibility—it does not care whether the input data is sorted or unsorted. However, its time complexity scales linearly **O(n)**,meaning performance drops dramatically as the size of the dataset expands, making it inefficient for large arrays.

B)BINARY SEARCH :-

Objective :- To implement the Binary Search algorithm in C++ using a sorted array, and to analyze its performance across best, worst, and average-case scenarios.

Binary Search Algorithm

Condition: The array must be sorted.

1. Start.
2. Read the sorted array and target element.
3. Set `low = 0`.
4. Set `high = n - 1`.
5. Calculate `mid`.
6. Compare `arr[mid]` with the target.
7. If equal, return the index.
8. If `arr[mid] < target`, search the right half.
9. Otherwise, search the left half.
10. Repeat until the element is found or `low > high`.
11. If not found, return `-1`.
12. Stop.

Program:

```
#include <iostream>
using namespace std;

// Function for Binary Search
// Requires sorted array
int binarySearch(int arr[], int size, int target) {

int low = 0;
int high = size - 1;

while (low <= high) {

int mid = low + (high - low) / 2;

if (arr[mid] == target) {
```

```
return mid;
}
else if (arr[mid] < target) {
low = mid + 1;
}
else {
high = mid - 1;
}
}

return -1;
}

int main() {

// Array must be sorted
int arr[] = {3, 7, 12, 23, 45, 56, 90};

int size = sizeof(arr) / sizeof(arr[0]);

int target;

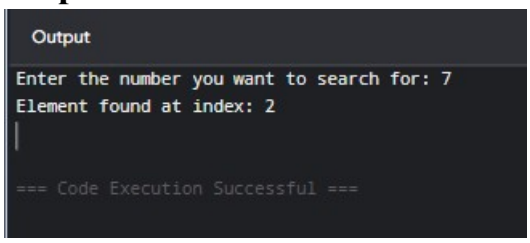
cout << "Enter the number you want to search for: ";
cin >> target;

int result = binarySearch(arr, size, target);

if (result != -1) {
cout << "Element found at index: " << result << endl;
}
else {
cout << "Element not found in the array." << endl;
}

return 0;
}
```

Output:



Analysis:-

A. Time Complexity Analysis

The execution time of Binary Search is determined by how many times the `while` loop runs to split the sorted array in half to pinpoint the target value.

- **Best Case: O(1)**

  This occurs when the target value happens to be exactly at the middle position (`arr[mid]`) on the very first calculation. The loop matches immediately and terminates after just 1 comparison.

- **Worst Case: O(log n)**

  This occurs if the target element is at the extreme far ends of the array or is completely missing. Because the algorithm continuously divides the remaining search zone into halves with every step (n/2,n/4), the total number of operations grows logarithmically with respect to the array size n.

- **Average Case: O(log n)**

  On average, most successful lookups or missed searches will find their answer deeper within the binary division tree structure, yielding an overall logarithmic growth rate.

B. Space Complexity Analysis

- **Space Complexity: O(1)**
  This iterative implementation utilizes constant auxiliary space. It updates a

  fixed set of tracking boundaries (`low`, `high`, `mid`) directly within the local memory stack. Since no dynamic arrays are created and no recursive functional call frames stack up, memory consumption never changes regardless of how large n becomes.

Conclusion:-

Through this practical, we observed that Binary Search is an exceptionally fast searching mechanism that runs with a logarithmic time efficiency **(O(log n)).** This makes it highly optimal for scaling across massive datasets where linear scanning becomes far too slow. The only trade-off is its rigid prerequisite constraint: the dataset **must be strictly pre-sorted** for the split-and-discard logic to work.